



Student Learning Management System Target Architecture

UPGRADE2ARCHITECT PHASE 3 PRESENTATION
MAY-2026

PRESENTER: PRATYUSH CHANDRA JHA

01 Use case details

02 Scope definition

03 Assumptions

04 Business requirements

05 Non-functional requirements

06 Architecture diagrams

07 Technology stack

08 AI usage & metrics

09 Advantages

10 Estimations & plan

11 Risks & mitigations

12 Links to HLD / LLD

13 Case studies

01

Use case details

Three user personas and their core needs

1. Use case details — three user personas

Use case

Admin

- ▶ Create, update, deactivate user accounts
- ▶ Assign roles (Admin / Instructor / Student)
- ▶ Approve courses before public catalogue listing
- ▶ View grade distribution and cohort-level reports
- ▶ Audit logs — filter by user, event type, date range

Instructor

- ▶ Create and publish courses with modules & lessons
- ▶ Upload video, PDF, and text content to S3
- ▶ Design timed quizzes and assignments
- ▶ Auto-grade MCQ/T-F; manually grade short answers
- ▶ View enrolled students and grade submissions

Student

- ▶ Search and enrol in free or paid courses
- ▶ Watch video lessons with chapter navigation
- ▶ Take timed quizzes with auto-submit on expiry
- ▶ Track completion % per course and per module
- ▶ Download auto-generated completion certificate

02

Scope definition

In scope for v1.0 and deferred to future phases

2. Scope definition — v1.0 vs future phases

Scope

In scope — v1.0

- ✓ User auth, RBAC (Admin / Instructor / Student)
- ✓ Course creation: modules, lessons (video/PDF/text)
- ✓ Timed quizzes, MCQ, T-F, short answer questions
- ✓ Auto-grading + manual grading with feedback
- ✓ Student progress tracking + completion percentage
- ✓ Certificate PDF auto-generation at 100% completion
- ✓ In-app + email notifications via SNS fanout
- ✓ Event audit logs filterable by Admin
- ✓ AWS ECS Fargate Docker containerised deployment
- ✓ 9-stage CI/CD pipeline with SonarQube quality gate

Out of scope — future phases

- OAuth 2.0 social login (AWS Cognito v2)
- Live virtual classroom / video conferencing
- Mobile native app (iOS / Android)
- AI-personalised learning path recommendations
- Third-party LTI integrations (Moodle, Canvas)
- Multi-tenancy / white-label branding
- Offline mode / progressive web app
- Advanced analytics with ML-based predictions
- Payment gateway marketplace
- SCORM / xAPI content standard support

03

Assumptions

Key architectural assumptions made during design

3. Assumptions

AWS cloud provider



All infrastructure on AWS. Multi-AZ in ap-south-1 (Mumbai) for low latency and data residency compliance.

Initial scale: 10K users



Supports 10,000 concurrent users at launch. Horizontal scaling beyond via ECS auto-scaling and DynamoDB on-demand.

Polyglot data strategy



RDS for ACID transactions, DynamoDB for high-write logs, Redis for low-latency cache, S3 for binary objects.

Java Spring Boot backend



All 8 microservices in Java 21 with Spring Boot 3.x. React/Next.js frontend SPA served via CloudFront CDN.

Custom JWT auth (v1)



Custom Spring Boot JWT Auth service for v1.0. AWS Cognito deferred to v2 for OAuth 2.0 and social login.

CI/CD from sprint 1



CodePipeline automated from day one. SonarQube warning mode for sprints 1–2, hard gate enforced from sprint 3.

04

Business requirements

F001–F011: functional requirements and feature map

4. Business requirements — feature map (F001–F011)

Requirements

F001	Responsive UI Dashboard, course player, timed quiz, search & filter, auto-refresh	F002	API integration S3 video streaming, email notifications, JSON responses, 10-min metadata cache	F003	Auth & RBAC JWT sessions, BCrypt passwords, 3-role access (Admin/Instructor/Student)	F004	User management Admin: full CRUD + roles. Instructor: own courses. Student: own profile
F005	Course management Lifecycle: Draft → Pending → Published → Archived. Admin approval workflow	F006	Assessments MCQ, T-F, short answer. Timed + MaxAttempts. Auto-grade + manual grade	F007	Progress tracking Completion % per course/module. Certificate at 100%. Cohort-level reports	F008	Grading & feedback Per-assessment + cumulative grades. Written feedback. Grade-release notification
F009	Notifications SNS fanout: in-app WebSocket + email SES. 48hr deadline reminders. Preferences	F010	Event logging All events logged: actor, type, description, timestamp. Admin filterable	F011	Code quality ESLint + SonarQube gates. 80% coverage. Unit tests for grading + progress + cert		

05

Non-functional requirements

Performance, security, reliability, and operational targets

5. Non-functional requirements

NFRs

≤30s

Content load SLA

80%

Min code coverage

99.9%

Uptime SLA target

2 AZ

High availability

Error handling

RFC 7807 error format.
GlobalExceptionHandler in every service.
CloudWatch alarms on error rate > 1%.

Rate limiting

AWS WAF: 100 req/min unauthenticated,
300 authenticated. 429 + Retry-After
header on throttle.

Content SLA ≤30s

CloudFront CDN at edge. S3 pre-signed
URLs bypass API for video. ElastiCache
Redis 10-min metadata cache.

Docker containers

All 8 services containerised. Images built
in CodeBuild, scanned in ECR, deployed
to ECS Fargate.

Modular + CI/CD

9-stage CodePipeline. SonarQube quality
gate. JaCoCo 80% coverage
enforcement. Blue/green deploy.

Scalability

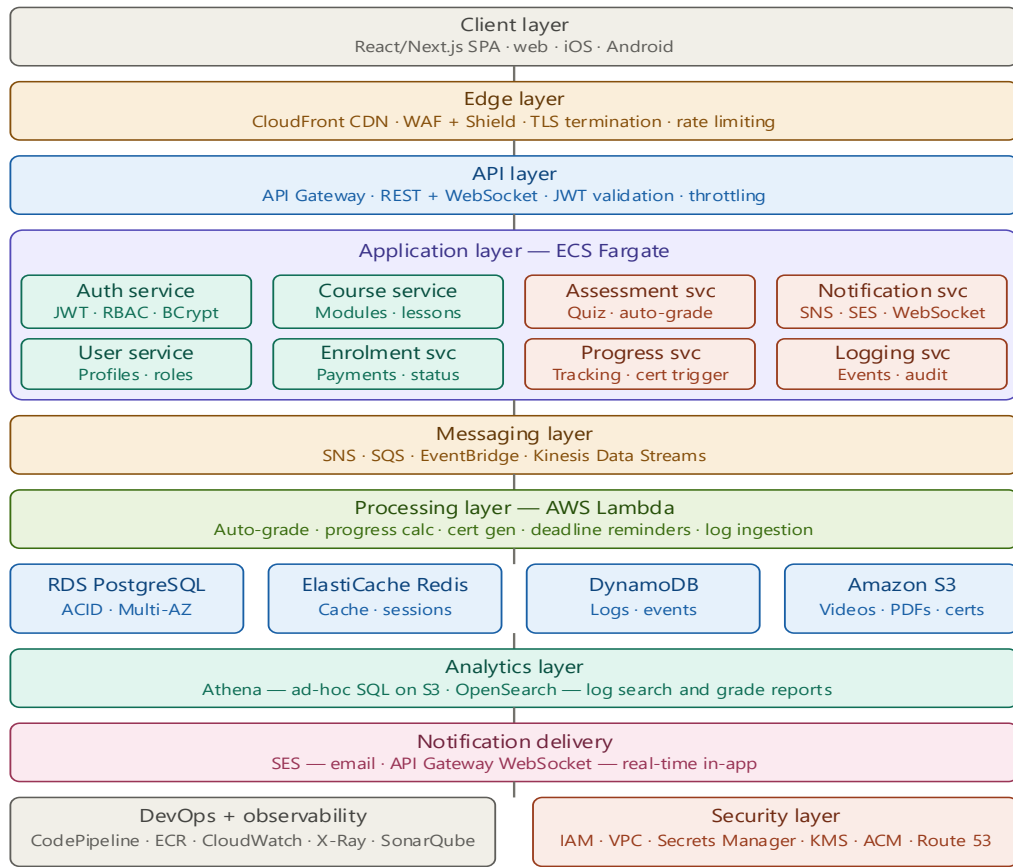
ECS auto-scaling on CPU > 60%.
DynamoDB on-demand. SQS absorbs
submission bursts without throttling.

06

Architecture diagrams

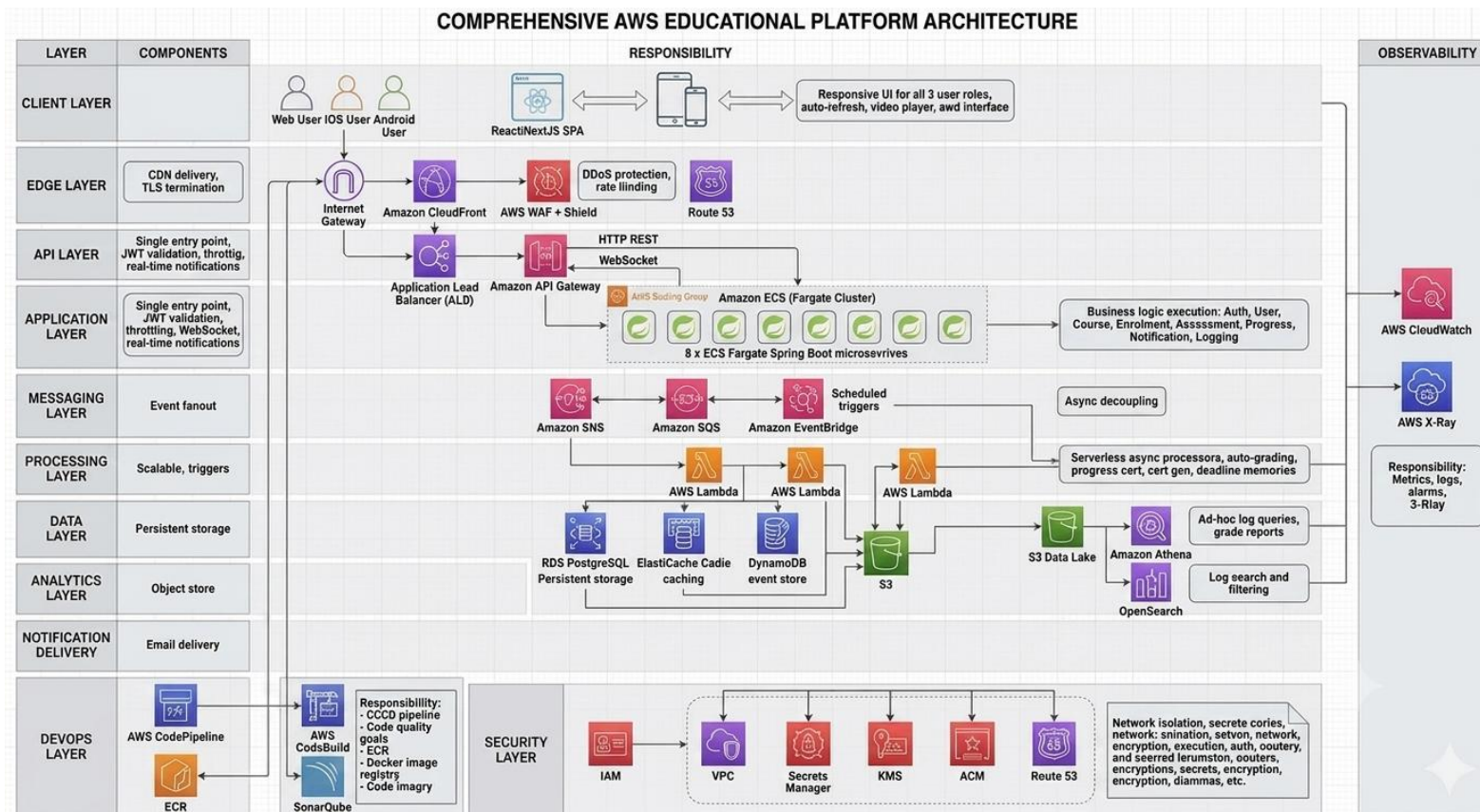
AWS microservices architecture and CI/CD pipeline

6a. AWS architecture — full stack view (logical)



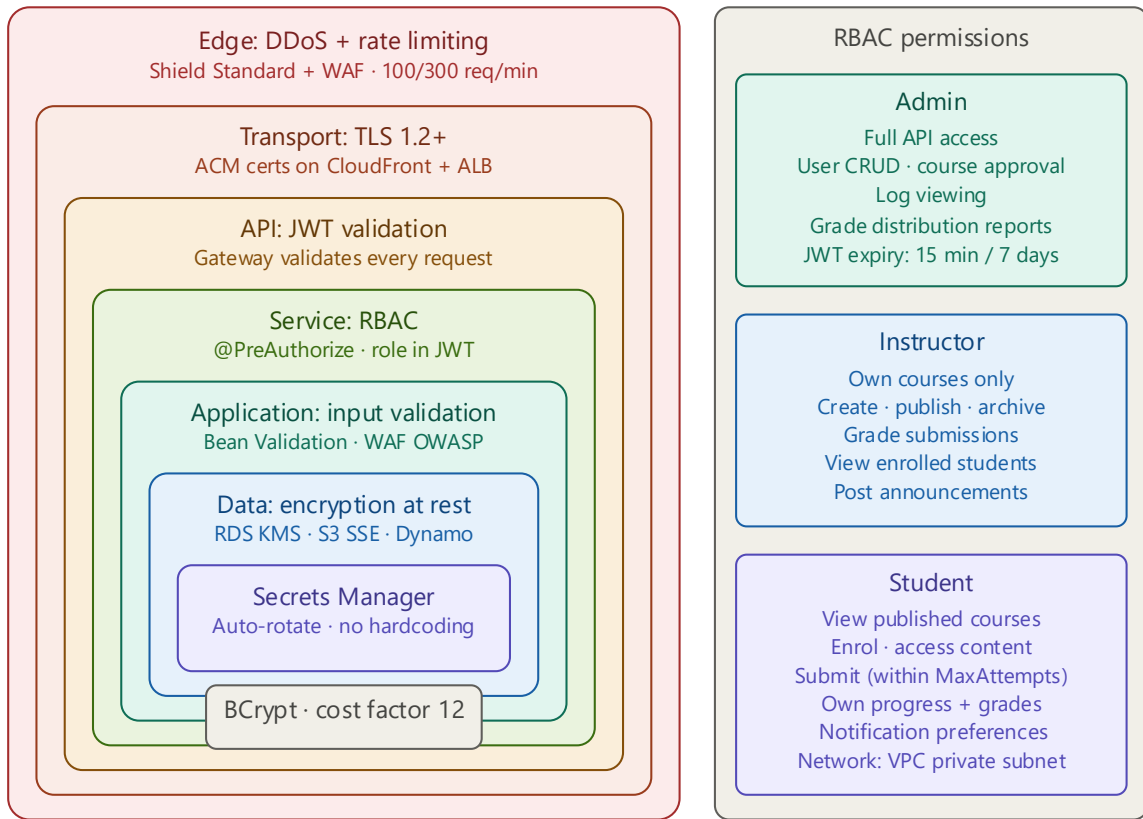
6a. AWS architecture — full stack view (AWS)

Architecture

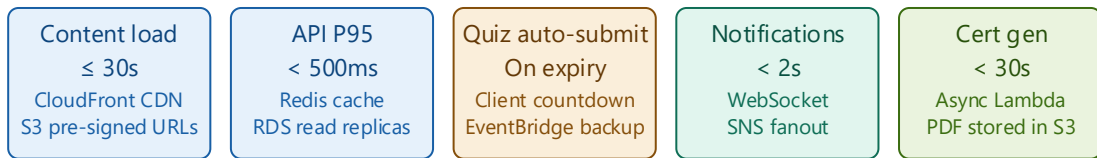


6b. AWS architecture — Security View

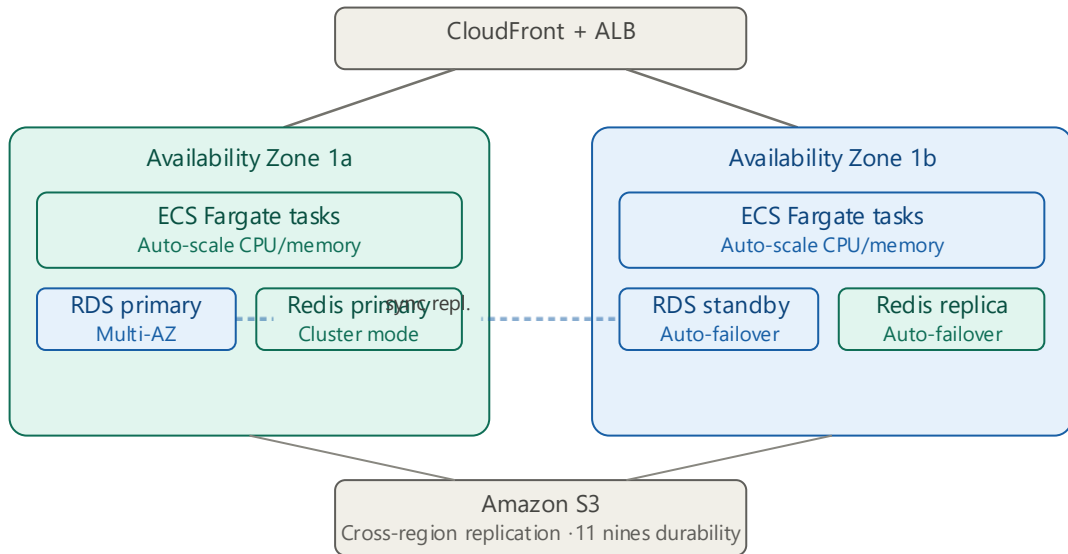
Architecture



6c. AWS architecture — High Availability



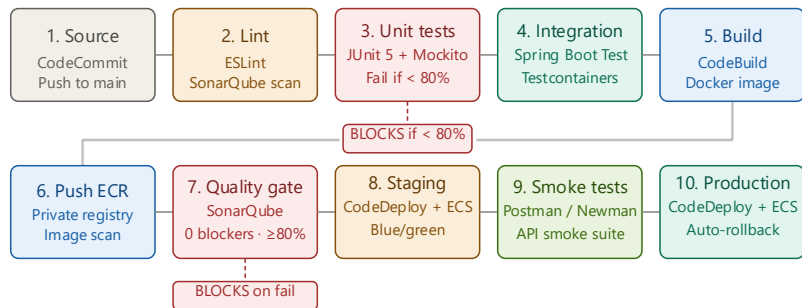
High availability — 2 availability zones



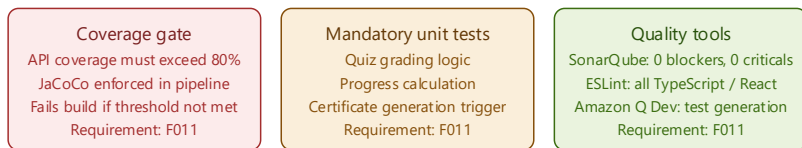
6d. CI/CD pipeline — 9-stage automated workflow

Pipeline stages

Push to main branch triggers all 10 stages automatically



Code quality standards (F011)



Blocking gate Deployment stage Test / verify stage Build / push stage Source / trigger

Every merge to main triggers all 9 stages — no manual gate to production

SonarQube blocks promotion if coverage < 80% or any critical/blocker issue

Blue/green ECS deployment — automatic rollback on CloudWatch alarm in < 5 min

ECR image scanning: CRITICAL vulnerability fails the push stage automatically

Amazon Q Developer accelerates unit test generation for grading and cert logic

07

Technology stack

AWS-native with Java Spring Boot, React, and Docker

7. Technology stack

Tech stack



Why ECS Fargate?

Serverless containers — no EC2 cluster management. Auto-scaling, blue/green deploys, IAM task roles. Aligns with Cognizant cloud-native practice.

Why polyglot data?

RDS for ACID, DynamoDB for high-write logs, Redis for cache, S3 for objects. Each store matched to its specific access pattern.

Why Spring Boot 3.x?

Mature enterprise ecosystem. Spring Security RBAC, Spring Data JPA, rich test tooling. Standard in Cognizant Java delivery practice.

Why React/Next.js?

Server-side rendering, static export to CloudFront CDN, built-in routing. React is the Cognizant frontend standard for web delivery.

08

AI usage & metrics

Amazon Q Developer and GenAI-assisted delivery

8. AI usage & metrics — Amazon Q Developer

AI usage

Unit test generation



GitHub Copilot auto-generates JUnit 5 tests for grading logic, progress calculation, and certificate trigger — saving ~40% of manual test-writing time.

Boilerplate acceleration



Spring Boot controllers, repository interfaces, and DTO mappers generated from entity descriptions. ~60% reduction in time spent writing boilerplate code.

Real-time code review



GitHub Copilot inline suggestions flag potential NPEs, missing @Transactional annotations, and Redis cache misuse as engineers write code in the IDE.

SonarQube remediation



AI-assisted remediation of code smells and security hotspots. Engineers resolve SonarQube findings 2× faster with GitHub Copilot context-aware suggestions.

40% Test time saved

60% Boilerplate saved

2× Faster fix cycle

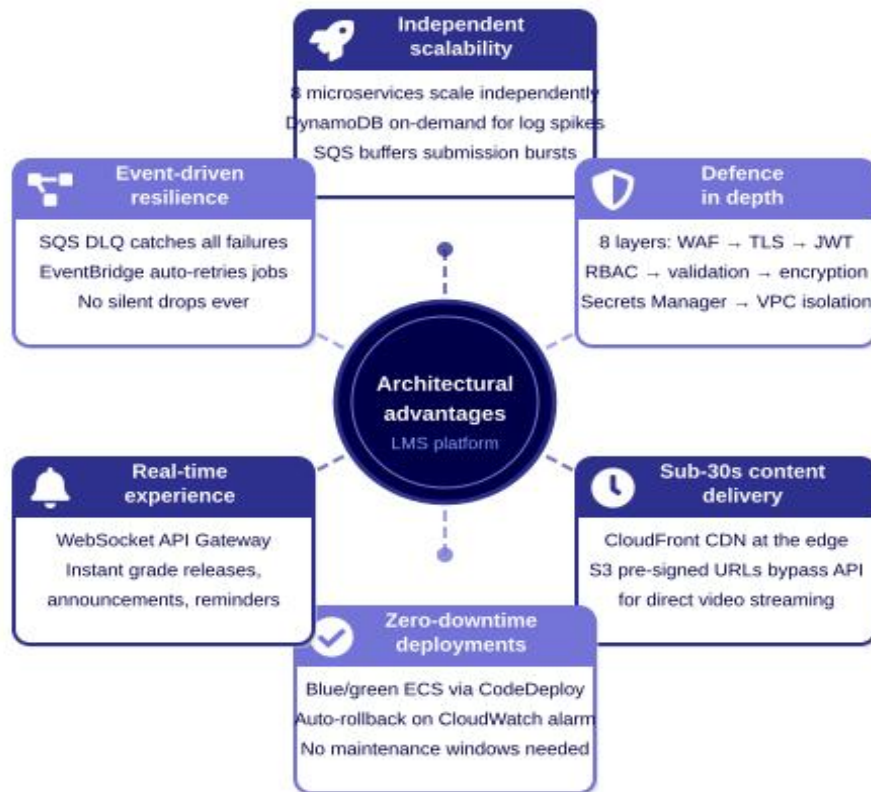
80% Coverage met

09

Advantages

Key architectural strengths of the proposed design

9. Architectural advantages



10

Estimations & plan

22-week delivery across 4 phases with milestones

10. Implementation plan — 4 phases / 22 weeks

Estimations

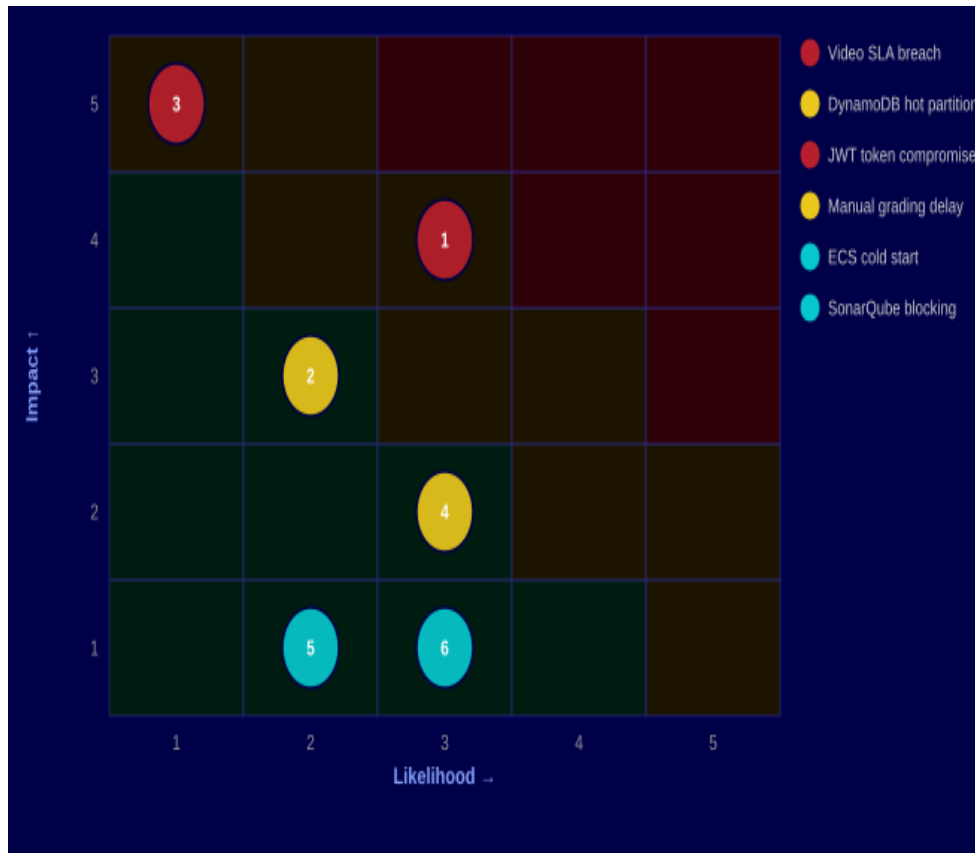


Component	Effort	Team
Auth + User services	2 sprints	2 BE devs
Course + Enrolment services	3 sprints	2 BE devs
Assessment + auto-grade Lambda	3 sprints	2 BE + 1 Lambda dev
Progress + certificate Lambda	2 sprints	1 BE + 1 Lambda dev
Notification service + WebSocket	2 sprints	1 BE dev
React frontend (all features)	5 sprints	3 FE devs
DevOps: CI/CD + monitoring	2 sprints	1 DevOps engineer
Security hardening + UAT	2 sprints	Full team

Risks & mitigations

Risk register with likelihood, impact, and mitigation plans

11. Risk register



1. Video SLA breach on slow connections

Adaptive bitrate (HLS). CloudFront edge caching. Multiple quality tiers.

2. DynamoDB hot partition on log writes

Partition key: actorId#date prefix. On-demand capacity absorbs write spikes.

3. JWT token compromise

15-min access TTL. Refresh token rotation. Redis blacklist on logout.

4. Manual grading SLA breach

Instructor alerts on ungraded submissions > 48hrs. Admin escalation report.

5. ECS cold start latency

Minimum 1 task always running per service. EventBridge pre-warm ping if needed.

6. SonarQube gate blocking releases

Warning-only mode for sprints 1–2. Hard gate enforced from sprint 3 onwards.

Red = High risk · Yellow = Medium · Teal = Low

12

Links to HLD & LLD

Supporting architecture documentation suite

12. Architecture documentation suite

[Links](#)



Architecture document

10 sections

Full AWS component reference, design decisions, scalability approach, security layers, operational runbook, and risk register.



High level design (HLD)

12 sections

System context, three-tier design, microservice map, event-driven notification flow, data architecture, NFR compliance.



Low level design (LLD)

11 Sections

ER diagram, class diagram, Spring Boot internals, quiz submission sequence, deployment topology, and state machines.



API design document

9 sections

All 37 REST endpoints with request/response schemas, JWT auth, RBAC table, RFC 7807 errors, WebSocket API, and file upload flow.

13

Case studies

AWS microservices at scale — industry validation

13. Case studies — industry references

Case studies

Coursera

100M+ learners

Architecture: AWS microservices + CloudFront CDN

Relevance to LMS:

S3 + CloudFront video streaming at scale, microservices decomposed by domain, event-driven certificate issuance — identical patterns to our LMS.

Duolingo

500M users

Architecture: AWS ECS + DynamoDB + Lambda

Relevance to LMS:

DynamoDB for high-write user activity events (lesson completions) — exactly the same pattern as our log and progress event store.

Khan Academy

135M registered

Architecture: AWS + React + CDN delivery

Relevance to LMS:

React SPA + CloudFront, async certificate generation, per-module progress tracking — architecture mirrors our LMS design.

Thank you

Questions & discussion

Supporting documents: [Architecture document](#) · [HLD](#) · [LLD](#) · [API design document](#)

